

LogoutButton]"

DASTAUTHREPORT: "false"

DASTREQUESTCOOKIES: "welcomebannerstatus:dismiss,cookieconsentstatus:dismiss"

DASTCRAWLGRAPH: "false"

> To highlight some variables, DASTTARGETURL and DASTAUTHURL provide a target for scanning and authentication. We provide credentials for authentication with DASTAUTHUSERNAME and DASTAUTHPASSWORD. The DASTAUTHUSERNAMEFIELD and related field variables tell DAST where to input login data. Our remaining settings are scan settings to help reduce the scan time for this demonstration.

1. After entering all of these values, your yaml file should look like this:

yaml

stages: List of stages for jobs, and their order of execution

- dast

include:

- template: DAST.gitlab-ci.yml

dast:

services:

- name: bkimminich/juice-shop:v16.0.0

alias: juiceshop

variables:

DASTTARGETURL: "http://juiceshop:3000/"

DASTAUTHURL: "http://juiceshop:3000/login"

DASTFULLSCAN: "false"

DASTAUTHUSERNAME: "admin@juice-sh.op"

DASTAUTHPASSWORD: "admin123" use protected/masked variables, this is only for

demonstration purposes

DASTAUTHUSERNAMEFIELD: "css:input[id=email]"

DASTAUTHPASSWORDFIELD: "css:input[id=password]"

DASTAUTHSUBMITFIELD: "css:button[id=loginButton]"

DASTSCOPEEXCLUDEELEMENTS: "css:[id=navbarLogoutButton]"

DASTAUTHREPORT: "false"

DASTREQUESTCOOKIES: "welcomebannerstatus:dismiss,cookieconsentstatus:dismiss"

DASTCRAWLGRAPH: "false"

1. Commit these changes and let the DAST scan run. You can monitor the job progress from Build > Pipelines.

> Note: This job can take up to 15 minutes to complete.

1. After this completes, in the left sidebar, select Secure > Vulnerability report.

1. Review the results found by the DAST scanner.

## Task B. Setting up API Scanners

API scanners allow you to scan your application API endpoints for potential vulnerabilities. To demonstrate this process, we will use an application template which contains an API configuration.

1. Navigate to your ILT group, which will be named My Test Group - XXXX, where the X's will be replaced with your username.

1. Select New project.

1. Select Create from template.

1. Select the Instance tab.

1. Select Use template next to the Security Essentials Labs template.

1. For Project name, input API Scanner Demo.

1. Select Create project.

1. Take some time here to review the postmancollection.json file. This file contains the definitions required to run API scanning against the application in this project. After reviewing the file and structure, you can proceed with enabling API scanning.

1. Create a .gitlab-ci.yml file by clicking on the + > New file and naming the file .gitlab-ci.yml.

1. To add API scanning to our container, define the dast stage and add the API security template.

```
yml
default:
  image: docker:26
```

```
include:
  - template: API-Security.gitlab-ci.yml
```

```
stages:
  - build
  - dast
```

```
variables:
  TARGETIMAGE: $CI_REGISTRYIMAGE/$CI_COMMITREFSLUG:$CI_COMMITSHA
```

1. Create a build job, which creates a Docker container to scan.

```
yml
build:
```

```
stage: build
services:
  - docker:26-dind
script:
  - docker build -t $TARGETIMAGE .
  - docker login -u "$CI_REGISTRYUSER" -p "$CI_REGISTRYPASSWORD" $CI_REGISTRY
  - docker push $TARGETIMAGE
```

1. Add the job definition for the API scanner.

```
yaml
apisecurity:
  services:
    - name: $CI_REGISTRYIMAGE/$CI_COMMITREFSLUG:$CI_COMMITSHA
      alias: target
  variables:
    APISECPOSTMANCOLLECTION: postmancollection.json
    APISECTARGETURL: http://target:7777
```

1. Commit these changes and view the results once the pipeline completes.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab6.md

> Estimated time to complete: 15 minutes

Objectives

Compliance scans allow you to control merge requests and security scans in your instances, groups, and projects. This lab will demonstrate how to enable security and merge request policies at a group level.

Task A. Setting up a Security Policy

1. Navigate to your group called My Test Group - XXXXX, where X is replaced by your username.

1. In the left sidebar, select Secure > Policies.

1. Select New policy.
1. Under Scan execution policy, select Select policy.
1. For the Name, enter Group security policy.
1. Ensure the Policy status is set to Enabled.
1. For Policy Scope, apply to all projects in this group without exceptions.
1. Under Actions, set the scan to run a SAST scan. Ensure that Runner Tags is set to selected automatically and Security job template is set to default.
1. Under Conditions, select Triggers. Have the policy run for all branches with No exceptions.
1. Select Configure with a merge request.
1. Select Merge to complete your configuration.

#### Task B. Scan Execution Policies in Action

1. Navigate to your group.
1. Select New project.
1. Select Create blank project.
1. In the Project name field input Security Compliance Testing.
1. Leave all other options as default and select Create project.
1. In the project repository, select + > New file.
1. In the Filename, input main.py.
1. Add the following code to the file:

```
py
print("Testing scanners!")
```
1. Commit your changes to the main branch.
1. In the left sidebar, select Build > Pipelines.
1. Select the pipeline that was created from your commit.
1. Review the jobs in the pipeline. Notice how a SAST scan is run on this commit even though no .gitlab-ci.yml configuration exists. Now, the projects in your group will always have security scans, even if they aren't defined in the .gitlab-ci.yml file yet.

## Task C. Merge Request Policies

With security scans in place, it would be ideal to be able to block merge requests that contain security vulnerabilities. In this section, you will create a merge request policy to achieve this goal.

1. Navigate to your group.

1. In the left sidebar, select Secure > Policies.

1. Select New policy.

1. Under Merge request approval policy, select Select policy.

1. In Name, enter the value Merge Request Security.

1. Under Policy status, ensure Enabled is selected.

1. For Policy scope, make sure the policy is applied to all projects in this group without exceptions.

1. Under Rules, set the scan type to Security Scan.

1. For the scanner, select All scanners.

1. Run the scan against all default branches.

1. Specify that the scan has No exceptions and finds Any vulnerabilities.

1. Set the severity level to All severity levels.

1. Set the status to New and All vulnerability states.

1. Under Actions, require 1 approval from the Owner role.

1. In Override project approval settings, make sure to uncheck the option Prevent approval by merge request's author and Prevent approval by commit author.

> We are only setting these options to off because we have one account in our group. For production environments, this is often toggled on.

1. Leave all other options as default and select Configure with a merge request.

1. Select Merge.

## Task D. Merge Request Policies in Action

1. Return to your group.

1. Select your Security Compliance Testing project.

1. In the left sidebar, select Code > Branches.
1. Select New branch.
1. Name the branch testpolicy, then select Create branch.
1. Select Create merge request.
1. Uncheck Mark as draft, then select Create merge request.
1. Select Code > Open in Web IDE.
1. Open your main.py file and add the following code to it:

```
py
import subprocess

in = input("Enter your server ip: ")
subprocess.run(["ping", in])

print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Select the source control icon, then select Commit and push to testpolicy'.
1. Return to your MR.
1. Allow your pipeline to complete. Once it completes, take note of the comment stating This merge request has policy violations and errors.
1. To resolve these issues, select Code > Open in Web IDE.
1. Open your main.py file and edit your main.py code to the following:

```
py
print("Attempting to connect to the server")
print("Application authentication was successful")
```

1. Select the source control icon, then select Commit and push to 'testpolicy.
1. Return to your MR.
1. Allow your pipeline to complete. Once complete, take note of the comment stating Security policy violations have been resolved.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

## Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandson.md

### GitLab System Administration Lab Guides

| Lab Name                      | Lab Link |
|-------------------------------|----------|
| Exploring the Admin Dashboard | Lab Link |
| Exploring the GitLab Instance | Lab Link |
| Backup and Restore            | Lab Link |
| Upgrading GitLab              | Lab Link |
| Logging and Monitoring        | Lab Link |
| Troubleshooting               | Lab Link |

### Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

### GitLab System Admin Course Description

## Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab System Administration, please submit your changes via Merge Request!

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonk8s.md

### GitLab System Administration Lab Guides

| Lab Name                      | Lab Link |
|-------------------------------|----------|
| Exploring the Admin Dashboard | Lab Link |
| Exploring the GitLab Instance | Lab Link |
| Backup and Restore            | Lab Link |
| Upgrading GitLab              | Lab Link |
| Logging and Monitoring        | Lab Link |
| Troubleshooting               | Lab Link |

### Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

[GitLab System Admin Course Description](#)

[Suggestions?](#)

If you wish to make a change to the Hands-On Guide for GitLab System Administration, please submit your changes via Merge Request!

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab1.md

## Objectives

In this lab, you will start to get familiar with the GitLab Admin dashboard. The Admin dashboard is one of the key areas administrators will use to manage their GitLab instances.

### Task A. Accessing the Admin Dashboard

1. Navigate to your GitLab instance.
1. Authenticate to the instance as your root admin user.
1. In the left sidebar, select Admin.
1. Take some time to explore the various sections of the Admin area.

### Task B. Configure Instance Sign Up and Sign In

To help secure your GitLab instance, you can customize the way users sign up and authenticate with your instance. In this task, you will enable two-factor authentication and Admin Mode. Admin Mode will introduce additional authentication for Admin tasks in your GitLab instance.

1. Navigate to the Admin area of your GitLab instance.
1. In the left sidebar, select Settings > General.
1. Select Expand next to Sign-up restrictions.
1. In Allowed domains for sign-ups, enter your company domain. This ensures only your company's users can register an account.
1. Select Save.
1. Select Expand next to Sign-in restrictions.
1. Check on Enforce two-factor authentication.



1. Check on Enable Admin Mode.
1. Select Save.
1. When you are navigated to the two-factor authentication page, select Configure Later.
1. To re-enter Admin Mode, select your profile image in the left sidebar. In the resulting menu, select Enter Admin Mode.
1. Input your Admin password in the prompt, then select Enter admin mode.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab1k8s.md

## Objectives

In this lab, you will start to get familiar with the GitLab Admin dashboard. The Admin dashboard is one of the key areas administrators will use to manage their GitLab instances.

### Task A. Accessing the Admin Dashboard

1. Navigate to your GitLab instance.
1. Authenticate to the instance as your root admin user.
1. In the left sidebar, select Admin.
1. Take some time to explore the various sections of the Admin area.

### Task B. Configure Instance Sign Up and Sign In

To help secure your GitLab instance, you can customize the way users sign up and authenticate with your instance. In this task, you will enable two-factor authentication and Admin Mode. Admin Mode will introduce additional authentication for Admin tasks in your GitLab instance.

1. Navigate to the Admin area of your GitLab instance.
1. In the left sidebar, select Settings > General.

1. Select Expand next to Sign-up restrictions.
1. In Allowed domains for sign-ups, enter your company domain. This ensures only your company's users can register an account.
1. Select Save.
1. Select Expand next to Sign-in restrictions.
1. Check on Enforce two-factor authentication.
1. Check on Enable Admin Mode.
1. Select Save.
1. When you are navigated to the two-factor authentication page, select Configure Later.
1. To re-enter Admin Mode, select your profile image in the left sidebar. In the resulting menu, select Enter Admin Mode.
1. Input your Admin password in the prompt, then select Enter admin mode.

## Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

## Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab2.md

## Objectives

In this lab, you will SSH into your GitLab instance and work with the various command line tools available in it. This will help you understand the different ways you can work with your GitLab installation.

### Task A. Accessing the GitLab Instance

You will access your GitLab installation using SSH. As a part of this class, you were provided an SSH key and instance IP to use for this process. To SSH into your GitLab instance, you can use the following steps.

1. On your local computer, open a terminal window.

1. Navigate to the directory that contains the SSH key file for your server.

1. SSH connections require that your private key file is not accessible by others. On Linux and MacOS, you can do this with the following command:

```
bash
chmod 400 <keyfilename>
```

On Windows, you can do this with the following command:

```
bash
icacls .\keyname.pem /inheritance:r
```

1. Use your assigned IP address and SSH key file to log in to the server that will host your GitLab Omnibus install:

```
bash
ssh -i <keyfilename> root@<vmipaddress>
```

> If you encounter an error like: WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!, you may need to reset your SSH known hosts. To do this, run the command `ssh-keygen -R <vmipaddress>`.

1. Press Enter.

1. If your system displays an authentication warning, type yes and press Enter.

1. After typing yes, you will be connected to your server.

## Task B. Interacting with gitlab-ctl

The gitlab-ctl command line utility provides you with tools for managing and troubleshooting your GitLab instance. In this task, you will see the different ways you can interact with your GitLab installation through the gitlab-ctl tool.

1. On your GitLab instance, run the command `sudo gitlab-ctl status`. You will see an output similar to below:

```
bash
run: alertmanager: (pid 904) 110s; run: log: (pid 899) 110s
run: gitaly: (pid 910) 110s; run: log: (pid 884) 110s
run: gitlab-exporter: (pid 903) 110s; run: log: (pid 886) 110s
run: gitlab-kas: (pid 893) 110s; run: log: (pid 881) 110s
run: gitlab-workhorse: (pid 894) 110s; run: log: (pid 888) 110s
run: logrotate: (pid 900) 110s; run: log: (pid 892) 110s
run: nginx: (pid 901) 110s; run: log: (pid 896) 110s
run: node-exporter: (pid 907) 110s; run: log: (pid 889) 110s
run: postgres-exporter: (pid 898) 110s; run: log: (pid 885) 110s
```

```
run: postgresql: (pid 890) 110s; run: log: (pid 882) 110s
run: prometheus: (pid 906) 110s; run: log: (pid 891) 110s
run: puma: (pid 905) 110s; run: log: (pid 895) 110s
run: redis: (pid 897) 110s; run: log: (pid 883) 110s
run: redis-exporter: (pid 902) 110s; run: log: (pid 887) 110s
run: sidekiq: (pid 911) 110s; run: log: (pid 909) 110s
```

> The status command shows you an overview of the GitLab components running on your server.

1. Next, try stopping a GitLab component, using the command `sudo gitlab-ctl stop nginx`.

> As an output, you will see `ok: down: nginx: 0s, normally up`.

1. To verify that the Nginx is no longer running, run the command `sudo gitlab-ctl status`. You will see that Nginx is not running in the output.

```
bash
down: nginx: 58s, normally up; run: log: (pid 896) 1070s
```

1. To restart Nginx, run the command `sudo gitlab-ctl restart nginx`.

> As an output, you will see `ok: run: nginx: (pid 1869) 0s`.

1. Run `sudo gitlab-ctl status` to verify that Nginx is up and running again.

### Task C. Interacting with GitLab Rails

GitLab Rails is a command line tool which allows you to interact with GitLab directly through Ruby on Rails. This tool can be valuable for troubleshooting and fixing instance errors.

1. To access GitLab Rails, run the command `sudo gitlab-rails console`.

> This command may take a few minutes to complete.

1. After the command completes, run `u = User.findbyusername('root')`.

> This command will find a user in GitLab with the username `root` and store them in the variable `u`.

1. Print the user's attributes using `pp u.attributes`.
1. Print the user's username using `pp u.username`.
1. Once complete, type `quit` to exit the Rails console.

### Task D. Interacting with PostgreSQL

GitLab uses PostgreSQL for all application related data. Sometimes it is helpful to view this

data through psql.

1. To access PostgreSQL, run the command `sudo gitlab-psql -d gitlabhqproduction`.

1. Run the command `\x` on to make queries results extended.

1. To see all the tables in the PostgreSQL database, run `\dt`.

1. Try querying some tables! For example: `SELECT * FROM projects;`, `SELECT * FROM users;`.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab2k8s.md

## Objectives

In this lab, you will SSH into your GitLab instance and work with the various command line tools available in it. This will help you understand the different ways you can work with your GitLab installation.

### Task A. Accessing the GitLab Instance

You will access your GitLab installation using SSH. As a part of this class, you were provided an SSH key and instance IP to use for this process. The instance you were provided has an active connection with helm to a Kubernetes cluster. To SSH into your instance, you can use the following steps.

1. On your local computer, open a terminal window.

1. Navigate to the directory that contains the SSH key file for your server.

1. SSH connections require that your private key file is not accessible by others. On Linux and MacOS, you can do this with the following command:

```
bash
chmod 400 <keyfilename>
```

On Windows, you can do this with the following command:

```
bash
icaccls .\keyname.pem /inheritance:r
```

1. Use your assigned IP address and SSH key file to log in to the server that will host your GitLab Omnibus install:

```
bash
ssh -i <keyfilename> root@<vmipaddress>
```

> If you encounter an error like: WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!, you may need to reset your SSH known hosts. To do this, run the command `ssh-keygen -R <vmipaddress>`.

1. Press Enter.

1. If your system displays an authentication warning, type yes and press Enter.

1. After typing yes, you will be connected to your server.

#### Task B. Interacting with kubectl

The kubectl command line tool is one of the main ways you will check and verify the status of your GitLab instance. In this section, we will explore the different ways we can see the status of a GitLab instance.

1. On your instance, run the command `sudo kubectl get pods`. You will see an output similar to below:

```
bash
NAME                                READY  STATUS   RESTARTS
AGE
gitlab-certmanager-7c799d587-9czfz    1/1    Running   0
4m5s
gitlab-certmanager-cainjector-9485f8595-976d7    1/1    Running   0
4m3s
gitlab-certmanager-webhook-dcf65786c-kfvhg    1/1    Running   0
4m3s
gitlab-gitaly-0                        1/1    Running   0
4m3s
gitlab-gitlab-exporter-76db6c9754-pcscv    1/1    Running   0
4m2s
gitlab-gitlab-shell-557686d7fc-x6rhm    1/1    Running   0
4m5s
gitlab-issuer-e45fa94-dp584            0/1    Completed  0
4m3s
gitlab-kas-5474bdbfdf-p6qw4            1/1    Running   4 (2m59s ago)
4m5s
gitlab-kas-5474bdbfdf-pzqwm            1/1    Running   4 (2m57s ago)
3m50s
```

|                                                 |     |           |   |
|-------------------------------------------------|-----|-----------|---|
| gitlab-migrations-92d5ea2-x6lc5                 | 0/1 | Completed | 1 |
| 4m2s                                            |     |           |   |
| gitlab-minio-5d646cdfb7-cdgw7                   | 1/1 | Running   | 0 |
| 4m5s                                            |     |           |   |
| gitlab-minio-create-buckets-5359da5-rccp6       | 0/1 | Completed | 0 |
| 4m3s                                            |     |           |   |
| gitlab-nginx-ingress-controller-5dd6f5f49-9k2cq | 1/1 | Running   | 0 |
| 4m5s                                            |     |           |   |
| gitlab-nginx-ingress-controller-5dd6f5f49-dgvft | 1/1 | Running   | 0 |
| 4m3s                                            |     |           |   |
| gitlab-postgresql-0                             | 2/2 | Running   | 0 |
| 4m3s                                            |     |           |   |
| gitlab-prometheus-server-c7f9d89dc-wz48w        | 2/2 | Running   | 0 |
| 4m5s                                            |     |           |   |
| gitlab-redis-master-0                           | 2/2 | Running   | 0 |
| 4m3s                                            |     |           |   |
| gitlab-registry-79f74497dd-bk8gw                | 1/1 | Running   | 0 |
| 4m3s                                            |     |           |   |
| gitlab-registry-79f74497dd-g9s84                | 1/1 | Running   | 0 |
| 3m50s                                           |     |           |   |
| gitlab-sidekiq-all-in-1-v2-54dfb45f84-d5gbq     | 1/1 | Running   | 0 |
| 4m2s                                            |     |           |   |
| gitlab-toolbox-9cdd66dc5-pz4gz                  | 1/1 | Running   | 0 |
| 4m2s                                            |     |           |   |
| gitlab-webservice-default-67f8cbfd9b-cmn7p      | 2/2 | Running   | 0 |
| 4m3s                                            |     |           |   |

> The status command shows you an overview of the GitLab components running on your server. Your component names may differ from the ones shown here.

1. Next, let's view details about a specific pod. To do this, run the command `kubectl describe pod <your-webservice-pod-name>`

> In this output, you will see in depth details about your webservice pod. This output is helpful for troubleshooting and monitoring your GitLab instance components.

### Task C. Working with helm

When you work with Kubernetes, many of your administration tasks will run through helm. To start, we will retrieve the values for our helm deployment, and make some small adjustments to the configuration.

1. To get your helm values, run the command `helm get values gitlab`. You will see an output of all your helm values.

1. Copy these into a file using the command `helm get values gitlab > values.yml`

1. Open the values.yml file in your text editor of choice.

1. Delete the top line written USER-SUPPLIED VALUES:.

1. Save the resulting file.

1. Let's try deploying our values file to verify that these values deploy successfully. To do this, run the command `helm upgrade gitlab gitlab/gitlab -f values.yml`.

> This command may take several minutes to run.

1. Once the command completes, run `kubectl get pods`. Verify that all pods complete deployment successfully.

#### Task D. Interacting with GitLab Rails

GitLab Rails is a command line tool which allows you to interact with GitLab directly through Ruby on Rails. This tool can be valuable for troubleshooting and fixing instance errors.

1. To access GitLab Rails, run the command `kubectl get pods -lapp=toolbox` to find your toolbox pod

1. Using your toolbox pod, launch an instance of gitlab-rails using the command `kubectl exec -it -c toolbox <toolbox-pod-name> -- gitlab-rails console`.

1. After the command completes, run `u = User.findbyusername('root')`.

> This command will find a user in GitLab with the username root and store them in the variable u.

1. Print the user's attributes using `pp u.attributes`.

1. Print the user's username using `pp u.username`.

1. Once complete, type `quit` to exit the Rails console.

#### Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

#### Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab3.md

> Estimated time to complete: 30 minutes



## Objectives

The objective of this lab is to demonstrate how to back up a GitLab instance on a virtual machine, and restore said instance to a previous state. For more information about backing up/restoring a GitLab instance, [click here](#).

### Task A. Configure backup settings

1. Open an SSH session on your GitLab instance server.

1. Search for the location of backup settings in `gitlab.rb`.

```
bash
sudo grep -n backuppath /etc/gitlab/gitlab.rb
```

1. Note the line number for the setting `gitlabrails['backuppath']`.

1. Create a new directory to hold GitLab backups.

```
bash
sudo mkdir /tmp/backups
```

1. Edit `gitlab.rb` to change the backup path. First, open the file in a text editor. For example: `sudo nano /etc/gitlab/gitlab.rb`.

1. Search for the line `gitlabrails['backuppath'] = "/var/opt/gitlab/backups"`. If you are using nano, you can press control-w to search.

1. Remove the `#` character in front of the `gitlabrails['backuppath']` line. This will uncomment the line, making it active.

1. Replace the path with `"/tmp/backups"`. The result is the following configuration line:

```
bash
gitlabrails['backuppath'] = "/tmp/backups"
```

1. Save the resulting file. If you are using nano, you can do this by pressing control-x.

1. Reconfigure to apply the changes.

```
bash
sudo gitlab-ctl reconfigure
```

### Task B. Backup the GitLab instance

1. Take a full backup of the GitLab instance.

```
bash
sudo gitlab-backup create
```

> In a production environment, you would also need to take a copy of the `/etc/gitlab/gitlab.rb` and `/etc/gitlab/gitlab-secrets.json`.

1. After the backup completes, go to the backup location and inspect the backup file.

```
bash
sudo ls /tmp/backups
sudo tar -tvf /tmp/backups/<backupfilename>
```

#### Task C. Make some changes to GitLab settings

1. Sign into your GitLab instance with a web browser and open your sidebar. In the bottom left corner, click Admin area.
2. In the left sidebar, select Settings > General.
3. Expand Account and limit and change the maximum attachment size to 500 MiB, and the default project limits to 10000.
4. Click Save changes to save the changes.
5. Refresh the page and verify your changes were applied.

#### Task D. Restore from backup

1. Return to the SSH session on your GitLab instance server.
1. Move your backup file to the location GitLab requires for performing the restore.

```
bash
sudo cp /tmp/backups/<backupfilename> /var/opt/gitlab/backups/
```

1. Ensure the backup file has correct permissions for performing the restore.

```
bash
sudo chown git:git /var/opt/gitlab/backups/<backupfilename>
```

1. Stop the puma and sidekiq services before restoring.

```
bash
sudo gitlab-ctl stop puma
sudo gitlab-ctl stop sidekiq
sudo gitlab-ctl status
```

1. Restore from backup. Replace <backuptimestamp> with the portion of the backup filename up to and including -ee. For example, if the backup file name starts with 16632077322022091515.3.3-ee, the command will be `sudo gitlab-backup restore BACKUP=16632077322022091515.3.3-ee`.

```
bash
sudo gitlab-backup restore BACKUP=<backuptimestamp>
```

1. Type yes when prompted during the restore operation. You may see what looks like error messages. That is normal.

1. When prompted to rebuild the authorizedkeys file, type yes.

1. Restart sidekiq and puma services.

```
bash
sudo gitlab-ctl start sidekiq
sudo gitlab-ctl start puma
sudo gitlab-ctl status
```

1. Wait up to 5 minutes before refreshing GitLab in your web browser. Verify that the maximum attachment size and the default project limits you changed revert back to the defaults (i.e. when the backup was taken).

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab3k8s.md

> Estimated time to complete: 30 minutes

Objectives

The objective of this lab is to demonstrate how to back up a GitLab instance on a virtual machine, and restore said instance to a previous state. For more information about backing up/restoring a GitLab instance, [click here](#).

Task A. Configure backup settings

1. Open an SSH session on your GitLab instance server.

1. Open your helm values using your text editor of choice.

1. Inside this file, you will need to update a few configurations to enable backups. To start, add the following global configuration:

```
yml
global:
  appConfig:
    artifacts:
      bucket: k8-lab-backup-bucket
    backups:
      bucket: k8-lab-backup-bucket
    lfs:
      bucket: k8-lab-backup-bucket
    packages:
      bucket: k8-lab-backup-bucket
    terraformState:
      bucket: k8-lab-backup-bucket
    tmpBucket:
      bucket: k8-lab-backup-bucket
    uploads:
      bucket: k8-lab-backup-bucket
```

> These configurations set the name of the remote storage bucket for each backup type.

1. Kubernetes backups are made through the toolbox pod. To enable this pod to run backups, it needs to be able to connect to the backup provider. The following configuration can be added to the gitlab block to achieve this:

```
yml
toolbox:
  backups:
    objectStorage:
      backend: s3
    config:
      key: config
      secret: my-s3cfg
```

1. To be able to access these buckets, you need to provide credentials for an AWS service account. To do this, run the command `kubectl create secret generic my-s3cfg --from-file=config=/tmp/my-s3cfg`.

1. After adding this configuration, run the command `helm upgrade --install gitlab gitlab/gitlab --timeout 100s -f values.yml`.

Task B. Backup the GitLab instance

1. To take a full backup, start by locating your toolbox pod.

```
bash
kubectl get pods -lapp=toolbox
```

1. Next, run the backup utility from the toolbox pod.

```
bash
kubectl exec <toolbox-name> -it -- backup-utility
```

1. Finally, retrieve your Kubernetes secrets and save them to a file.

```
bash
Get your rails secrets
kubectl get secrets | grep rails-secret
```

Save the secrets to a local location

```
kubectl get secrets <rails-secret-name> -o jsonpath="{.data['secrets\.yaml']}" | base64
--decode > gitlab-secrets.yaml
```

At this point, the backup is now stored in your object storage. Take note of the file name, for example, s3://bucket/17292610402024101817.4.1-ee.gitlabbackup.tar

Important: Take note of the timestamp (in this example: 17292610402024101817.4.1-ee). You will need this for the restore process.

#### Task C. Make some changes to GitLab settings

1. Sign into your GitLab instance with a web browser and open your sidebar. In the bottom left corner, click Admin area.
2. In the left sidebar, select Settings > General.
3. Expand Account and limit and change the maximum attachment size to 500 MiB, and the default project limits to 10000.
4. Click Save changes to save the changes.
5. Refresh the page and verify your changes were applied.

#### Task D. Restore from backup

1. Return to the SSH session on your GitLab instance server.

1. Delete your current Kubernetes secrets:

```
bash
```

```
kubectl delete secret <rails-secret-name>
```

1. Create a new set of secrets based on your backed up Kubernetes secrets:

```
bash
kubectl create secret generic <rails-secret-name> --from-file=secrets.yml=gitlab-
secrets.yml
```

1. Restart your Kubernetes pods to apply the secret

```
bash
kubectl delete pods -lapp=sidekiq,release=gitlab
kubectl delete pods -lapp=webservice,release=gitlab
kubectl delete pods -lapp=toolbox,release=gitlab
```

1. To start the restore process, locate the name of your toolbox pod.

```
bash
kubectl get pods lapp=toolbox
```

1. Scale your Kubernetes pod replicas down to 0

```
bash
kubectl scale deploy -lapp=sidekiq,release=gitlab --replicas=0
kubectl scale deploy -lapp=webservice,release=gitlab --replicas=0
kubectl scale deploy -lapp=prometheus,release=gitlab --replicas=0
```

1. Run a restore using the timestamp ID you received from the backup command

```
bash
kubectl exec <Toolbox pod name> -it -- backup-utility --restore -t your-timestamp-id
```

> If you are prompted to start the restore process, type yes.

1. After the restore process completes, scale your pods back up:

```
bash
kubectl scale deploy -lapp=sidekiq,release=gitlab --replicas=1
kubectl scale deploy -lapp=webservice,release=gitlab --replicas=1
kubectl scale deploy -lapp=prometheus,release=gitlab --replicas=1
```

1. Try to access your Kubernetes instance and verify that the restore completed successfully.

## Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

### Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab4.md

> Estimated time to complete: 30 minutes

### Objectives

The objective of this lab is to demonstrate how to upgrade a GitLab instance.

#### Task A. View your current GitLab version

1. Navigate to your GitLab instance and sign in as the root user.

1. In the left sidebar, select Help > Help.

1. At the top of the page, you will see your full GitLab version. For this lab, you will see 17.4.1-ee.

#### Task B. View the available GitLab versions

Updates to GitLab Omnibus are completed through your package manager, by installing the Omnibus package that corresponds to your GitLab version. To view the versions of GitLab available for your package, you can SSH onto your instance and view package versions for your package manager.

1. SSH into your GitLab instance.

1. Run the command `sudo apt-get upgrade` to update your packages.

1. Run the command `sudo apt-cache madison gitlab-ee`.

1. For this lab, locate version 17.5.0-ee.0. Take note of this version, we will be upgrading to this version of GitLab.

#### Task C. Run the upgrade

With our target GitLab version identified, we can now run an upgrade of GitLab.

1. If you haven't already, SSH onto your GitLab instance.

1. Take a backup of your instance using `sudo gitlab-backup create`.
1. Navigate to <https://gitlab-com.gitlab.io/support/toolbox/upgrade-path/>.
1. Input your current GitLab version (17.4.1) and your target version (17.5.5).
1. Ensure that Edition is set to Enterprise, and Distro is set to Ubuntu.
1. Select Go!.

In the upgrade path, you will see only a single step is required, which is to install 17.5.0. Let's complete this process.

1. On your GitLab instance, run the command `sudo apt-get install gitlab-ee=17.5.0-ee.0`.
1. Wait for the installation to complete. Once it does, navigate to your instance in your web browser.

You should be met with a message HTTP 502: Waiting for GitLab to boot. Wait at this screen until the boot completes and you are navigated back to your GitLab instance.

#### Task D. Validate the upgrade

At this point, you have been redirected to the login page of your instance. To validate your upgrade, complete the following steps:

1. Authenticate to your GitLab instance as the root user.
1. In the left sidebar, select Admin.
1. In the left sidebar, select Monitoring > Background migrations.
1. Review the table for any migrations in the Queued section. These migrations need to complete before you complete any additional upgrades. Once they complete, they will show in the Finished tab.

> As an example, after this upgrade you will likely see the migration `BackfillIssuesCorrectWorkItemTypeId`: issues. Try to find this migration in either the Queued or Finished tab.

1. To validate your version, in the left sidebar, select Help. In the resulting menu, you should now see version 17.5.

#### Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

#### Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please



submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab4k8s.md

> Estimated time to complete: 30 minutes

## Objectives

The objective of this lab is to demonstrate how to upgrade a GitLab instance.

### Task A. View your current GitLab version

1. Navigate to your GitLab instance and sign in as the root user.

1. In the left sidebar, select Help > Help.

1. At the top of the page, you will see your full GitLab version. For this lab, you will see 17.4.1-ee.

### Task B. View the available GitLab versions

Updates to GitLab Omnibus are completed through your package manager, by installing the Omnibus package that corresponds to your GitLab version. To view the versions of GitLab available for your package, you can SSH onto your instance and view package versions for your package manager.

1. SSH into your GitLab instance.

1. Run the command `helm repo update` to update your helm charts.

1. Run the command `helm search repo gitlab/gitlab --versions`.

1. For this lab, locate version 17.5.0-ee.0 (associated with helm chart version 8.5.0). Take note of this version, we will be upgrading to this version of GitLab.

### Task C. Run the upgrade

With our target GitLab version identified, we can now run an upgrade of GitLab.

1. If you haven't already, SSH onto your GitLab instance.

1. Take a backup of your instance using the following commands:

```
bash
find the toolbox pod
kubectl get pods -lapp=toolbox
```

Run the backup utility  
`kubectl exec <toolbox-name> -it -- backup-utility`

Get your rails secrets  
`kubectl get secrets | grep rails-secret`

Save the secrets to a local location  
`kubectl get secrets <rails-secret-name> -o jsonpath="{.data['secrets\.yaml']}" | base64 --decode > gitlab-secrets.yaml`

1. Navigate to <https://gitlab-com.gitlab.io/support/toolbox/upgrade-path/>.

1. Input your current GitLab version (17.4.1) and your target version (17.5.0).

1. Ensure that Edition is set to Enterprise, and Distro is set to Ubuntu.

1. Select Go!.

In the upgrade path, you will see only a single step is required, which is to install 17.5.0. Let's complete this process.

1. On your GitLab instance, run the command:

```
bash
helm upgrade gitlab gitlab/gitlab \
--version 8.5.0 \
-f gitlab.yaml
```

1. Wait for the installation to complete. Once it does, navigate to your instance in your web browser.

You should be met with a message HTTP 502: Waiting for GitLab to boot. Wait at this screen until the boot completes and you are navigated back to your GitLab instance.

#### Task D. Validate the upgrade

At this point, you have been redirected to the login page of your instance. To validate your upgrade, complete the following steps:

1. Authenticate to your GitLab instance as the root user.

1. In the left sidebar, select Admin.

1. In the left sidebar, select Monitoring > Background migrations.

1. Review the table for any migrations in the Queued section. These migrations need to complete before you complete any additional upgrades. Once they complete, they will show in the Finished tab.

> As an example, after this upgrade you will likely see the migration `BackfillIssuesCorrectWorkItemTypeId`: issues. Try to find this migration in either the Queued or Finished tab.

1. To validate your version, in the left sidebar, select Help. In the resulting menu, you should now see version 17.5.

## Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

## Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab5.md

> Estimated time to complete: 30 minutes

## Objectives

The objective of this lab is to demonstrate how to use Prometheus and Grafana for log analysis and monitoring.

## Task A. Access the Prometheus Service

1. GitLab's Prometheus server can be reached via TCP port 9090. Unfortunately, the training environment currently blocks inbound traffic to that port. As a workaround, you can open an SSH tunnel as follows.

```
bash
ssh -L 9090:localhost:9090 -i <SSHKEYNAME> root@<GITLABINSTANCEHOSTNAME>
```

1. Navigate to `http://localhost:9090` in a web browser to view the built-in Prometheus server.

1. To view the available metrics in Prometheus, navigate to `http://localhost:9090/metrics`.

> To see a more detailed list of each metric, check out the documentation.

## Task B. Viewing Prometheus Metrics

> For this example, suppose you wanted to monitor HTTP requests sent into your GitLab instance. To achieve this, we can use some built in GitLab metrics.

1. Navigate to `http://localhost:9090`.

1. In the input beside the magnifying glass button, type `httprequeststotal`.

1. Click the Execute button.

1. Click Table to view a table of all the results generated by the query.

> You will likely see a variety of different HTTP requests in the results. Note that you can see the job, instance, HTTP method and status of each request.

1. Click Graph to view a graph of your data.

> On the left hand side of the graph, you will see a - and + button. You can click these buttons to change the scale of the graph. Try adjusting the scale to see how it impacts your data.

### Task C. Alert Configurations

> Prometheus can be used to alert administrators based on specific metrics. GitLab comes with a set of predefined alerts for Prometheus.

1. Navigate to `http://localhost:9090`.

1. Click Alerts in the top menu.

> Note that there are two sets of alert rules, one named GitLab and one named Node. The GitLab alerts correspond to platform issues such as a service being down, Postgres being down, or high queueing within services. A Node alert corresponds to the server GitLab is deployed on.

1. Note the file that contains the alerting rules. By default, the file is `/var/opt/gitlab/prometheus/rules/gitlab.rules`.

> As an administrator, you can define custom rules inside this file to use for alerting. This will allow you to monitor your system for any potential issues.

### Task D. Common Performance Metrics

There are a variety of different metrics administrators can use to monitor the performance of their GitLab instances. Let's examine a few common queries you can use to monitor an instance.

1. Navigate to `http://localhost:9090`.

1. In the input beside the magnifying glass button, type the following query:

text

$((\text{nodememoryMemAvailablebytes} / \text{nodememoryMemTotalbytes}) \text{ or } ((\text{nodememoryMemFreebytes} + \text{nodememoryBuffersbytes} + \text{nodememoryCachedbytes}) / \text{nodememoryMemTotalbytes})) \text{ } 100$

> This query will show you the percentage of memory available on your instance.

1. Click Graph to view the percentage of memory over time.

1. In the query input, delete your previous query and replace it with the following query:

text

```
1 - avg without (mode,cpu) (rate(nodecpusecondstotal{mode="idle"}[5m]))
```

> This query will show you the percentage of CPU used on your instance.

1. Click Graph to view the percent of CPU used over time.

> For these metrics and many others, you may want to consider alerting for specific conditions. For example, you could alert for cases where the CPU usage exceeds 50% consistently, as it might indicate an issue.

## Task B. Install Grafana

Grafana provides you with a method of visualizing logs and metrics for your GitLab instance. Let's start by installing Grafana

1. SSH into your GitLab instance.

1. Pull the Grafana key to your instance using `wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor > /etc/apt/keyrings/grafana.gpg`.

1. Add the repository to your package manager using `echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | tee /etc/apt/sources.list.d/grafana.list`.

1. Update the apt repository: `sudo apt-get update`.

1. Install Grafana and its related components using `apt-get install loki promtail grafana`.

1. Start the Grafana server using `sudo systemctl start grafana-server`.

1. To verify that Grafana is running, navigate to `http://your-ip:3000/login`. Your default username and password are admin.

## Task C. Configure Grafana to collect logs

Grafana provides you with useful tools for log collection and analysis. Let's see how Grafana can be configured to capture and display some of our GitLab log files.

1. SSH onto your GitLab instance.

1. Open the file `/etc/promtail/config.yml` in the text editor of your choice.

1. In this file, verify your server, positions, and clients:

```

yml
server:
  httplistenport: 9080
  grpclistenport: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://localhost:3100/loki/api/v1/push

```

1. Take note specifically of the loki URL and port. You will use this later.

1. Log collection configurations are set in the scrapeconfigs section of the configuration file. These configurations require the following values to be set:

- targets: The location of the target log file, usually localhost.
- job: A unique name for the log scraping job.
- path: The location of the log files.

1. As an example, copy the following scrapeconfigs into your Grafana configuration file, replacing the existing scrapeconfigs file.

```

yml
scrapeconfigs:
  - jobname: system
  staticconfigs:
    - targets:
        - localhost
      labels:
        job: nginx
        path: /var/log/gitlab/nginx/
    - targets:
        - localhost
      labels:
        job: workhorse
        path: /var/log/gitlab/gitlab-workhorse/
    - targets:
        - localhost
      labels:
        job: rails
        path: /var/log/gitlab/gitlab-rails/productionjson.log

```

> This configuration adds three log files to Grafana: Nginx, Workhorse, and rails.

1. After adding this data, save the file. Restart promtail using `sudo systemctl restart promtail`.

1. With this complete, navigate to Grafana at <http://your-gitlab-ip:3000/login>.

1. Authenticate as your admin user.

1. In the left sidebar, select Connections > Data Sources.

1. Select Loki.

1. In the URL, input <http://localhost:3100>.

1. Select Save & Test.

1. You should see the message Data source successfully connected.

Task D. Analyze data with Grafana

1. Navigate to your Grafana instance.

1. In the left sidebar, select Explore.

1. Make sure Loki is selected in the dropdown beside Outline.

1. In the Label filters enter job.

1. In the value, enter nginx.

> You can replace this with any job name defined in your configuration file if you wish.

1. Select Run query to see your results. Take some time here to review the logs and options for filtering logs in Grafana.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab5k8s.md

> Estimated time to complete: 30 minutes

Objectives

The objective of this lab is to demonstrate how to use Prometheus and Grafana for log analysis

and monitoring.

#### Task A. Access the Prometheus Service

1. GitLab's Prometheus server can be reached via TCP port 9090. Unfortunately, the training environment currently blocks inbound traffic to that port. As a workaround, you can open an SSH tunnel as follows.

```
bash
ssh -L 8001:localhost:8001 -i <SSHKEYNAME> root@<GITLABINSTANCEHOSTNAME>
```

#### Task B. Access the Prometheus Service

1. GitLab's Prometheus server can be reached via TCP port 9090. To access this port, you need to create a proxy into your Kubernetes cluster. This can be done with the following command:

```
bash
kubectrl proxy
```

1. To access Prometheus, you need to know its pod name. To find this, run the command `kubectrl get pods`. Search for Prometheus in the output.

1. Navigate to

`http://localhost:8001/api/v1/namespaces/default/pods/PROMETHEUSPODNAME:9090/proxy/` in a web browser to view the built-in Prometheus server.

1. To view the available metrics in Prometheus, navigate to

`http://localhost:8001/api/v1/namespaces/default/pods/PROMETHEUSPODNAME:9090/proxy/metrics`.

> To see a more detailed list of each metric, check out the documentation.

#### Task C. Viewing Prometheus Metrics

> For this example, suppose you wanted to monitor HTTP requests sent into your GitLab instance. To achieve this, we can use some built in GitLab metrics.

1. Navigate to

`http://localhost:8001/api/v1/namespaces/default/pods/PROMETHEUSPODNAME:9090/proxy/`.

1. In the input beside the magnifying glass button, type `httprequeststotal`.

1. Click the Execute button.

1. Click Table to view a table of all the results generated by the query.

> You will likely see a variety of different HTTP requests in the results. Note that you can see the job, instance, HTTP method and status of each request.



1. Click Graph to view a graph of your data.

> On the left hand side of the graph, you will see a - and + button. You can click these buttons to change the scale of the graph. Try adjusting the scale to see how it impacts your data.

#### Task D. Alert Configurations

> Prometheus can be used to alert administrators based on specific metrics. GitLab comes with a set of predefined alerts for Prometheus.

1. Navigate to

<http://localhost:8001/api/v1/namespaces/default/pods/PROMETHEUSPODNAME:9090/proxy/>.

1. Click Alerts in the top menu.

> Note that there are two sets of alert rules, one named GitLab and one named Node. The GitLab alerts correspond to platform issues such as a service being down, Postgres being down, or high queueing within services. A Node alert corresponds to the server GitLab is deployed on.

1. Note the file that contains the alerting rules. By default, the file is `/var/opt/gitlab/prometheus/rules/gitlab.rules`.

> As an administrator, you can define custom rules inside this file to use for alerting. This will allow you to monitor your system for any potential issues.

#### Task E. Common Performance Metrics

There are a variety of different metrics administrators can use to monitor the performance of their GitLab instances. Let's examine a few common queries you can use to monitor an instance.

1. Navigate to

<http://localhost:8001/api/v1/namespaces/default/pods/PROMETHEUSPODNAME:9090/proxy/>.

1. In the input beside the magnifying glass button, type the following query:

text

$$((\text{nodememoryMemAvailablebytes} / \text{nodememoryMemTotalbytes}) \text{ or } ((\text{nodememoryMemFreebytes} + \text{nodememoryBuffersbytes} + \text{nodememoryCachedbytes}) / \text{nodememoryMemTotalbytes})) \text{ } 100$$

> This query will show you the percentage of memory available on your instance.

1. Click Graph to view the percentage of memory over time.

1. In the query input, delete your previous query and replace it with the following query:

text

`1 - avg without (mode,cpu) (rate(nodecpusecondstotal{mode="idle"}[5m]))`

> This query will show you the percentage of CPU used on your instance.

1. Click Graph to view the percent of CPU used over time.

> For these metrics and many others, you may want to consider alerting for specific conditions. For example, you could alert for cases where the CPU usage exceeds 50% consistently, as it might indicate an issue.

## Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

## Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab6.md

> Estimated time to complete: 30 minutes

## Objectives

The objective of this lab is to demonstrate how to use Prometheus and Grafana for log analysis and monitoring.

### Task A. Issues at the reverse proxy

1. From a shell session on your GitLab instance, view one of the NGINX active logs.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

1. After typing this, navigate to your GitLab instance and try accessing various pages on the instance. Note how each request has an Nginx log message associated with it.

1. Let's cause an error in the Nginx service. Stop the Nginx service using the following command:

```
bash
sudo gitlab-ctl stop nginx
```

1. Attempt to navigate to `http://<yourgitlabinstance>` using a web browser. Your web browser should display "This site can't be reached" or a similar message.

1. Check nginx/accesslog again.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

> Take note that no new log messages have appeared in these logs. Since Nginx cannot be reached, it cannot log the access request. In this case, we can immediately understand that Nginx has an issue.

1. Verify web services aren't running or listening anywhere.

```
bash
curl -i http://localhost/nginxstatus
curl -i http://localhost:80
```

1. Restart NGINX services.

```
bash
sudo gitlab-ctl restart nginx
```

1. View the logs and verify that Nginx is now receiving and processing requests.

```
bash
sudo gitlab-ctl tail nginx/gitlabaccess.log
```

1. Verify the webserver is running and listening on port 80.

```
bash
curl -i http://localhost/nginxstatus
```

## Task B. Tracing Issuing at the GitLab Rails level

For this example, let's suppose that an issue occurred in GitLab Rails. To simulate this:

1. Stop the puma service by running `sudo gitlab-ctl stop puma`.

1. Navigate to your GitLab instance. Note that you get a 502 Waiting for GitLab to boot error.

1. The first location that requests reach for this component is Nginx. Let's take a look at the logs by running:

```
bash
cat /var/log/gitlab/nginx/gitlabaccess.log
```

1. In this output, search for your HTTP request. It will look something like this:

```
bash
173.34.175.144 - - [25/Oct/2024:14:48:00 +0000] "GET / HTTP/1.1" 502 2026
"http://34.56.107.198/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10157) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36" -
```

> We can tell this is the request based on the URL being accessed and the Nginx status code. This status code is a 502 and the request is to the root of the web application.

1. This output tells us our request reaches Nginx. From here, we can trace to the next location in the architecture, Workhorse. To view the logs for Workhorse, run:

```
bash
cat /var/log/gitlab/gitlab-workhorse/current
```

1. In this output, look for the request that matches the URL of your Nginx request. It will look like this:

```
json
{"backendid":"rails","contenttype":"text/html; charset=utf-
8","correlationid":"01JB22H7ENN72DH5XNMTB2170Z","durationms":0,"host":"34.56.107.198","level":"
info","method":"GET","msg":"access","proto":"HTTP/1.1","referrer":"http://34.56.107.198/","remo
teaddr":"173.34.175.144:0","remoteip":"173.34.175.144","route":"","routeid":"default","status":
502,"system":"http","time":"2024-10-
25T14:50:49Z","ttfbms":0,"uri":"/favicon.ico","useragent":"Mozilla/5.0 (Macintosh; Intel Mac OS
X 10157) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0
Safari/537.36","writtenbytes":2026}
```

> In this request, we can collect some additional important details. We can see our request was sent to rails. If the request reached rails, we can use the correlationid to locate it in the logs. Let's take note of the correlation ID and search for it in the Rails log.

1. To view the log for your Rails application, run the command:

```
bash
cat /var/log/gitlab/gitlab-rails/productionjson.log
```

1. These logs have a lot of data, since we are looking for a correlation ID, we can try grepping for it in the logs:

```
bash
cat /var/log/gitlab/gitlab-rails/productionjson.log | grep your-correlation-id
```

1. You will get no results. This tells you the GitLab Rails did not receive the request from

Workhorse. From here, we can diagnose the issue with rails, starting by looking at the status:

```
bash
sudo gitlab-ctl status
```

1. In this output, we will see that Rails is not running. To remedy this, we can restart the Rails process:

```
bash
sudo gitlab-ctl restart puma
```

### Task C. Tracing a Healthy Request

With GitLab Rails back up, we can now try tracing a valid request to see how it looks in the logs. This will help you get comfortable with correlating a correlation ID to a request.

1. Make a request to your GitLab instance.

1. Locate the request correlation ID by running:

```
bash
cat /var/log/gitlab/gitlab-workhorse/current
```

> In the results, look for the request that matches your request URL. It should be the most recent request. Take note of the correlationid property.

1. Search for the correlation ID in the GitLab Rails logs:

```
bash
cat /var/log/gitlab/gitlab-rails/productionjson.log | grep your-correlation-id
```

1. As a result, you should see a log message that matches your correlation ID, like this:

```
json
{"method":"GET","path":"/","format":"html","controller":"RootController","action":"index","status":200,"time":"2024-10-25T15:05:43.089Z","params":[],"correlationid":"01JB23CG5BM7X1KDDKX5DZ2RCK","meta.callerid":"RootControllerindex","meta.featurecategory":"groupsandprojects","meta.remoteip":"173.34.175.144","meta.user":"root","meta.userid":1,"meta.clientid":"user/1","remoteip":"173.34.175.144","userid":1,"username":"root","ua":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10157) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36","queuedurations":0.036988,"requesturgency":"low","targetdurations":5,"redis calls":68,"redisallowedcrossslotcalls":5,"redis durations":0.012599,"redisreadbytes":7048,"rediswritebytes":5562,"redis cachecalls":25,"redis cacheallowedcrossslotcalls":3,"redis cachedurations":0.004691,"redis cachereadbytes":1971,"redis cachewritebytes":2244,"redisdbloadbalancingcalls":3,"redisdbloadbalancingdurations":0.000548,"redisdbloadbalancingwritereadbytes":104,"redisfeatureflagcalls":31,"redisfeatureflagdurations":0.005347,"redisfeaturefla
```

```
greadbytes":4694,"redisfeatureflagwritebytes":2526,"redisrepositorycachecalls":3,"redisreposito
rycachedurations":0.000853,"redisrepositorycachereadbytes":203,"redisrepositorycachewritebytes"
:79,"redisessionscalls":6,"redisessionsallowedcrossslotcalls":2,"redisessionsdurations":0.00
116,"redisessionsreadbytes":180,"redisessionswritebytes":609,"dbcount":52,"dbwritecount":0,"d
bcachedcount":11,"dbtxncount":0,"dbreplicatxncount":0,"dbprimarytxncount":0,"dbmaintxncount":0,
"dbcitxncount":0,"dbmainreplicatxncount":0,"dbcireplicatxncount":0,"dbreplicacount":0,"dbprimar
ycount":52,"dbmaincount":52,"dbcicount":0,"dbmainreplicacount":0,"dbcireplicacount":0,"dbreplic
awritecount":0,"dbprimarywritecount":0,"dbmainwritecount":0,"dbciwritecount":0,"dbmainreplicawr
itecount":0,"dbcireplicawritecount":0,"dbreplicacachedcount":0,"dbprimarycachedcount":11,"dbmai
ncachedcount":11,"dbcicachedcount":0,"dbmainreplicacachedcount":0,"dbcireplicacachedcount":0,"d
breplicawalcount":0,"dbprimarywalcount":0,"dbmainwalcount":0,"dbciwalcount":0,"dbmainreplicawal
count":0,"dbcireplicawalcount":0,"dbreplicawalcachedcount":0,"dbprimarywalcachedcount":0,"dbmai
nwalcachedcount":0,"dbciwalcachedcount":0,"dbmainreplicawalcachedcount":0,"dbcireplicawalcached
count":0,"dbreplicatxnmaxdurations":0.0,"dbprimarytxnmaxdurations":0.0,"dbmaintxnmaxdurations":
0.0,"dbcitxnmaxdurations":0.0,"dbmainreplicatxnmaxdurations":0.0,"dbcireplicatxnmaxdurations":0
.0,"dbreplicatxndurations":0.0,"dbprimarytxndurations":0.0,"dbmaintxndurations":0.0,"dbcitxndur
ations":0.0,"dbmainreplicatxndurations":0.0,"dbcireplicatxndurations":0.0,"dbreplicadurations":
0.0,"dbprimarydurations":0.039,"dbmaindurations":0.039,"dbcidurations":0.0,"dbmainreplicadurati
ons":0.0,"dbcireplicadurations":0.0,"cpus":0.225071,"memobjects":91945,"membytes":8675485,"memm
allocs":19041,"memtotalbytes":12353285,"pid":239709,"workerid":"puma2","ratelimitinggates":[],"
dbdurations":0.02908,"viewdurations":0.11937,"durations":0.20995}
```

When Rails is accepting requests, these log messages provide a wealth of information for troubleshooting and debugging.

#### Task D. Collecting SOS

In some cases, troubleshooting will require the assistance of GitLab Support. To help GitLab Support troubleshoot any errors, it is helpful to provide a full account of your instance logs. For this purpose, you can use the GitLab SOS tool.

1. To run GitLab SOS, use the following commands:

```
bash
/opt/gitlab/embedded/bin/git clone --recursive https://gitlab.com/gitlab-
com/support/toolbox/gitlabsos.git && cd gitlabsos
sudo /opt/gitlab/embedded/bin/ruby ./gitlabsos.rb
```

1. After the run completes, you will see the location of your support bundle, in the form of `/root/gitlabsos/gitlabsos.your-instance-namegitly-nginx-psql-puma-redis-sidekiq.tar.gz`.

> When you file support requests, you can attach this bundle to provide all of your instance logs.

#### Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

## Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

---

## SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/sysadminhandsonlab6k8s.md

> Estimated time to complete: 30 minutes

## Objectives

The objective of this lab is to demonstrate how to use Prometheus and Grafana for log analysis and monitoring.

### Task A. Issues at the reverse proxy

1. From a shell session on your GitLab instance, view one of the NGINX active logs.

```
bash
kubectl get pods
kubectl logs <webservice-pod-name>
```

1. After typing this, navigate to your GitLab instance and try accessing various pages on the instance. Note how each request has an Nginx log message associated with it.

1. Let's cause an error in the Nginx service. Stop the Nginx service using the following command:

```
bash
kubectl scale deploy -lapp=webservice,release=gitlab --replicas=0
```

1. Attempt to navigate to `http://<yourgitlabinstance>` using a web browser. Your web browser should display "This site can't be reached" or a similar message.

1. Check `nginx/accesslog` again.

```
bash
kubectl logs <webservice-pod-name>
```

> Take note that no new log messages have appeared in these logs. Since Nginx cannot be reached, it cannot log the access request. In this case, we can immediately understand that Nginx has an issue.

1. Verify web services aren't running or listening anywhere.

```
bash
curl -i http://localhost/nginxstatus
curl -i http://localhost:80
```

1. Restart NGINX services.

```
bash
kubectl scale deploy -lapp=webservice,release=gitlab --replicas=1
```

1. View the logs and verify that Nginx is now receiving and processing requests.

```
bash
kubectl logs <webservice-pod-name>
```

1. Verify the webserver is running and listening on port 80.

```
bash
curl -i http://localhost/nginxstatus
```

Task B. Tracing Issuing at the GitLab Rails level